



Islamic University of Technology

Department of Computer Science and Engineering (CSE)

B.Sc. in Software Engineering

SWE 4802: Software Maintenance Lab

Summer, 2024-2025

Lab Project

Software Maintenance

Namisa Najah Raisa

210042112

Program: B.Sc in SWE

Department of CSE

Date: August 17, 2026

Contents

1	Project selection	3
2	Lab 01 : Program Slicing and analysis	3
2.1	Static Slicing	3
2.2	Dynamic Slicing	4
2.3	Static analysis	4
2.3.1	AST generation	4
2.3.2	Metrics Calculation	5
2.4	Dynamic analysis	7
3	Lab 02 : CPG	8
3.1	Overview	8
3.2	Approach	8
3.3	Implementation	8
3.4	Benefits for Software Maintenance	10
4	Lab 03 : Usage of the tools in the project	11
4.1	Loguru	11
4.1.1	Usage in project	11
4.1.2	Insight from output	12
4.1.3	Maintenance use case	12
4.2	Pysnooper	13
4.2.1	Usage in project	13
4.2.2	Insight from output	13
4.2.3	Maintenance use case	13
4.3	CProfile	13
4.3.1	Usage in project	13
4.3.2	Insight from output	14
4.3.3	Maintenance use case	14
4.4	Snakeviz	14
4.4.1	Usage in project	14
4.4.2	Insight from output	15
4.4.3	Maintenance use case	15
4.5	Viztracer	15
4.5.1	Usage in project	15
4.5.2	Insight from output	16
4.5.3	Maintenance use case	16
5	Lab 04: Reverse Engineering and Re-engineering	16
5.1	Reverse Engineering	16
5.1.1	Tools Used	16
5.1.2	Methodology	16
5.1.3	Results	17
5.1.4	Bytecode Disassembly Output	17
5.1.5	Class Diagram	18
5.1.6	Module Dependency / Cross-Reference Diagram	19

5.1.7	Conclusion	19
6	Lab 05: Code Clone Detection and Analysis	19
6.1	Rsearch code clone detection tools	19
6.1.1	Description	19
6.1.2	Why I selected jscpd	20
6.2	Methodology	20
6.3	Results	21
6.3.1	Overall Statistics	21
6.4	Analysis	21
6.4.1	Locations	21
6.4.2	Clone Types	22
6.4.3	Impact on Maintainability	22
6.5	Proposed Solution	22
7	Software Maintenance	23
7.1	Corrective Maintenance	23
7.1.1	Feasibility Study	23
7.1.2	Program Comprehension	23
7.1.3	Change Management	24
7.1.4	Impact Analysis	24
7.1.5	Reverse Engineering	25
7.1.6	Refactoring (Implementation)	25
7.2	Adaptive Maintenance	25
7.2.1	Feasibility Study	26
7.2.2	Program Comprehension	26
7.2.3	Change Management	26
7.2.4	Impact Analysis	27
7.2.5	Reverse Engineering	27
7.2.6	Refactoring (Implementation)	27
7.3	Preventive Maintenance	28
7.3.1	Feasibility Study	28
7.3.2	Program Comprehension	29
7.3.3	Change Management	29
7.3.4	Impact Analysis	30
7.3.5	Reverse Engineering	30
7.3.6	Refactoring (Implementation)	30
7.4	Perfective Maintenance	32
7.4.1	Feasibility Study	32
7.4.2	Program Comprehension	32
7.4.3	Change Management	33
7.4.4	Impact Analysis	33
7.4.5	Reverse Engineering	33
7.4.6	Refactoring (Implementation)	34

1 Project selection

For this Software Maintenance Lab, I used a Student Management System written in Python as my base project. It's a console-based CRUD application - you can add, search, update, remove, sort, and view students, plus generate a full report with stats and GPA-based classification.

The architecture is layered:

- **models/student.py** - the Student dataclass with a `classification()` method that buckets GPA into Distinction/Merit/Pass/At Risk
- **services/student_service.py** - CRUD operations and JSON persistence
- **services/report_service.py** - statistics, classification summaries, formatted reports
- **utils/validators.py** - input validation
- **utils/logger_setup.py** - centralized Loguru configuration
- **main.py** - the menu-driven entry point

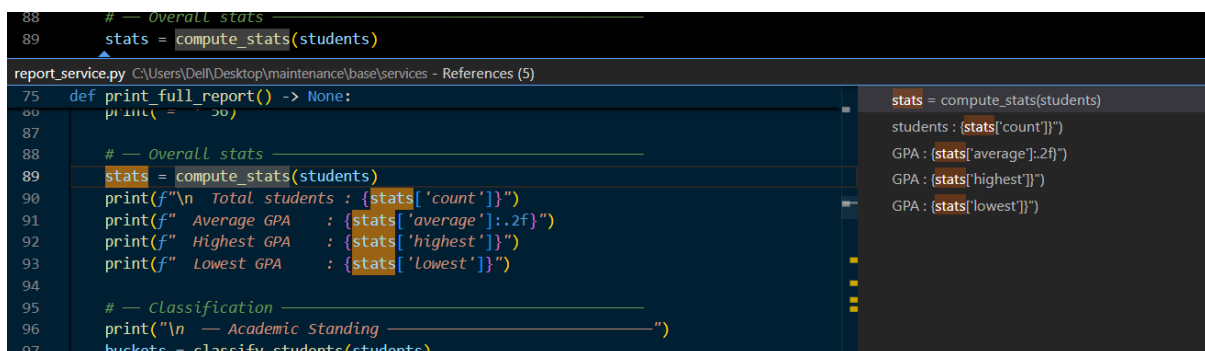
Git link: [Github Repository](#)

2 Lab 01 : Program Slicing and analysis

2.1 Static Slicing

Computed purely from the source code, without running the program. It answers: “*which statements could possibly affect this variable, over all possible inputs and execution paths?*”

1. Open `services/report_service.py` and place the cursor on `stats` inside the `print_full_report()` function.
2. Right-click `stats` → **Go to Definition**. This jumps to `statistics = compute\_stats(students)`.
3. Right-click `compute_stats` → **Go to Definition** to open its body in `report_service.py`.



```
88 # -- Overall stats --
89 stats = compute_stats(students)

report_service.py C:\Users\Dell\Desktop\maintenance\base\services - References (5)
75 def print_full_report() -> None:
86     print(...)
87
88 # -- Overall stats --
89 stats = compute_stats(students)
90 print(f"\n Total students : {stats['count']}")
91 print(f" Average GPA      : {stats['average']:.2f}")
92 print(f" Highest GPA       : {stats['highest']}")
93 print(f" Lowest GPA        : {stats['lowest']}")
94
95 # -- Classification --
96 print("\n Academic Standing")
97 buckets = classify_students(students)
```

stats = compute_stats(students)
students : {stats['count']}
GPA : {stats['average']:.2f}
GPA : {stats['highest']}
GPA : {stats['lowest']}

Figure 1: static slicing

2.2 Dynamic Slicing

Computed from an actual execution trace of the program with a specific input. It answers: “which statements actually did affect this variable, in this one run?”

1. Open the Run and Debug panel[click the play/bug icon in the left sidebar (or Ctrl+Shift+D).]
2. Set a breakpoint at your slicing criterion. Open report_service.py, click in the left margin next to the line:


```
1 stats = compute_stats(students)
2
```
3. Start debugging - press F5 (or click ”Run and Debug” → ”Python File”). Run the program as normal, choose option 2 to add a student, then option 6 to trigger the report. Execution will pause at the breakpoint.
4. Step into the function - press F11 (Step Into) on the compute_stats(students) line.
5. Step over each line with F10, watching the Variables panel (left side, under ”Locals”) update live - we’ll see students, grades, and stats get their actual values at each step. This is the dynamic slice: only the lines that actually ran, with the actual data.

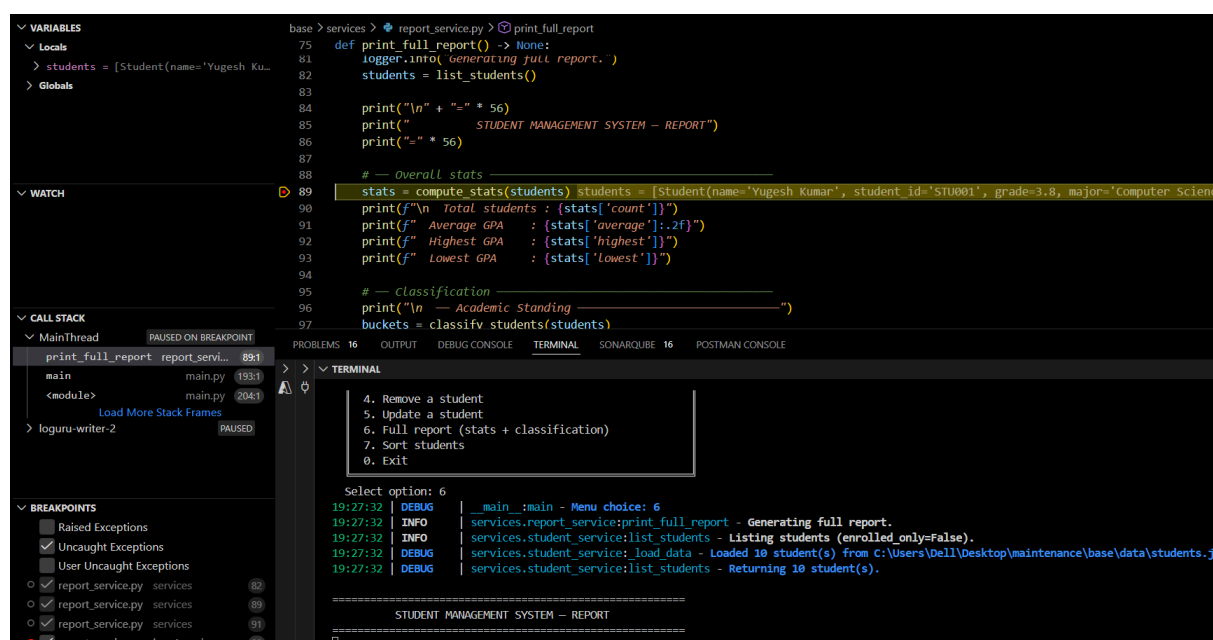


Figure 2: dynamic slicing

2.3 Static analysis

2.3.1 AST generation

1. Created a small script *ast_analysis.py* in my project root.
2. Run it:

```
1 python ast_analysis.py > ast_output.txt
2
```



```

(.venv) PS C:\Users\Dell\Desktop\maintenance\base> radon cc . -a -s
main.py
  F 123:0 action_update - A (5)
  F 139:0 action_sort - A (5)
  F 162:0 main - A (5)
  F 65:0 _print_students - A (3)
  F 86:0 action_add - A (2)
  F 111:0 action_remove - A (2)
  F 49:0 _print_menu - A (1)
  F 78:0 action_view - A (1)
  F 102:0 action_search - A (1)
models\student.py
  M 22:4 Student.classification - A (4)
  C 12:0 Student - A (3)
  M 56:4 Student.__str__ - A (2)
  M 33:4 Student.to_dict - A (1)
  M 45:4 Student.from_dict - A (1)
services\report_service.py
  F 75:0 print_full_report - B (8)
  F 18:0 compute_stats - A (3)
  F 45:0 classify_students - A (2)
  F 64:0 major_breakdown - A (2)
services\student_service.py
  F 141:0 update_student - B (7)
  F 25:0 _load_data - A (4)
  F 54:0 list_students - A (4)
  F 98:0 search_student - A (4)
  F 115:0 remove_student - A (4)
  F 68:0 add_student - A (3)
  F 42:0 _save_data - A (2)
  F 177:0 sort_by_name - A (1)
  F 182:0 sort_by_grade - A (1)
utils\logger_setup.py
  F 18:0 setup_logging - A (2)
  F 62:0 get_logger - A (1)
utils\validators.py
  F 14:0 validate_name - A (4)
  F 45:0 validate_grade - A (3)
  F 61:0 validate_major - A (3)
  F 73:0 validate_menu_choice - A (3)
  F 31:0 validate_student_id - A (2)

34 blocks (classes, functions, methods) analyzed.
Average complexity: A (2.911764705882353)

```

Figure 4: Cyclomatic complexity per function

```

Average complexity: 1.7 (2151178/12500255)
(.venv) PS C:\Users\Dell\Desktop\maintenance\base> radon mi . -s
ast_analysis.py - A (100.00)
disassemble.py - A (100.00)
main.py - A (62.12)
docs\source\conf.py - A (100.00)
models\student.py - A (76.16)
services\report_service.py - A (65.15)
services\student_service.py - A (59.82)
utils\logger_setup.py - A (100.00)
utils\validators.py - A (53.77)

```

Figure 5: Maintainability Index score per file

2.4 Dynamic analysis

1. Install *coverage.py*.
2. Run the program under coverage instead of plain Python. In the terminal (from the project root, so main.py's path setup still works):

```

1 coverage run main.py
2

```

3. Generate a text summary:

```

1 coverage report -m
2

```

This prints a table per file: % Covered and the Missing line numbers (lines that never ran in this session).

4. Generate an HTML report for a visual, file-by-file view:

```

1 coverage html
2

```

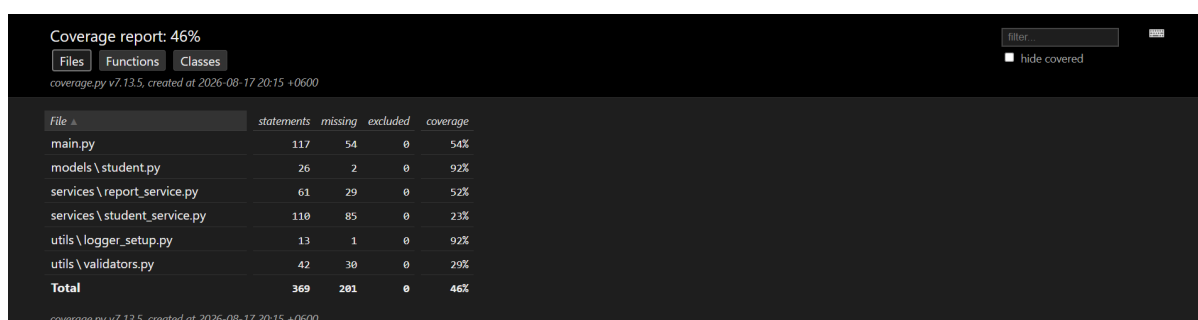


Figure 6: Code Coverage

3 Lab 02 : CPG

3.1 Overview

A **Code Property Graph (CPG)** is a graph-based representation of source code that combines multiple program representations into a single structure. Rather than storing the program as plain text, a CPG represents source code as interconnected nodes and edges describing its syntax, control flow, and data flow. This unified representation enables efficient static program analysis, vulnerability detection, dependency analysis, and software maintenance tasks.

The three primary components integrated into a CPG are:

- **Abstract Syntax Tree (AST):** Represents the syntactic structure of the program.
- **Control Flow Graph (CFG):** Represents the possible execution paths through the program.
- **Program Dependence Graph (PDG):** Represents data and control dependencies between statements.

By combining these graphs into a single property graph, developers can perform complex code queries without repeatedly parsing the source code.

3.2 Approach

I adapted the workflow to work with a Python program taken from my Student Management System project.

The overall workflow followed these steps:

1. Configured a Neo4j AuraDB instance to store graph data.
2. Installed Java and the Joern framework inside Google Colab.
3. Created a source directory and wrote the Python program into `/content/src/program.py`.
4. Used Joern to parse the source code and generate a Code Property Graph (CPG).
5. Exported the generated graph and prepared it for import into Neo4j for visualization and querying.

3.3 Implementation

The notebook first created a temporary source directory and stored the program to be analyzed. After installing Joern and its dependencies, the parser was executed on the source directory to generate the Code Property Graph.

The generated CPG contains graph nodes representing program entities such as methods, variables, literals, identifiers, control structures, and function calls. These nodes are connected by different edge types describing syntactic relationships, execution flow, and data dependencies.

The generated graph can then be imported into Neo4j, allowing Cypher queries to explore relationships between different parts of the program.

- All nodes and edges (overview)



Figure 7: Nodes and edges

- AST (Abstract Syntax Tree)
c
- CFG (Control Flow Graph)

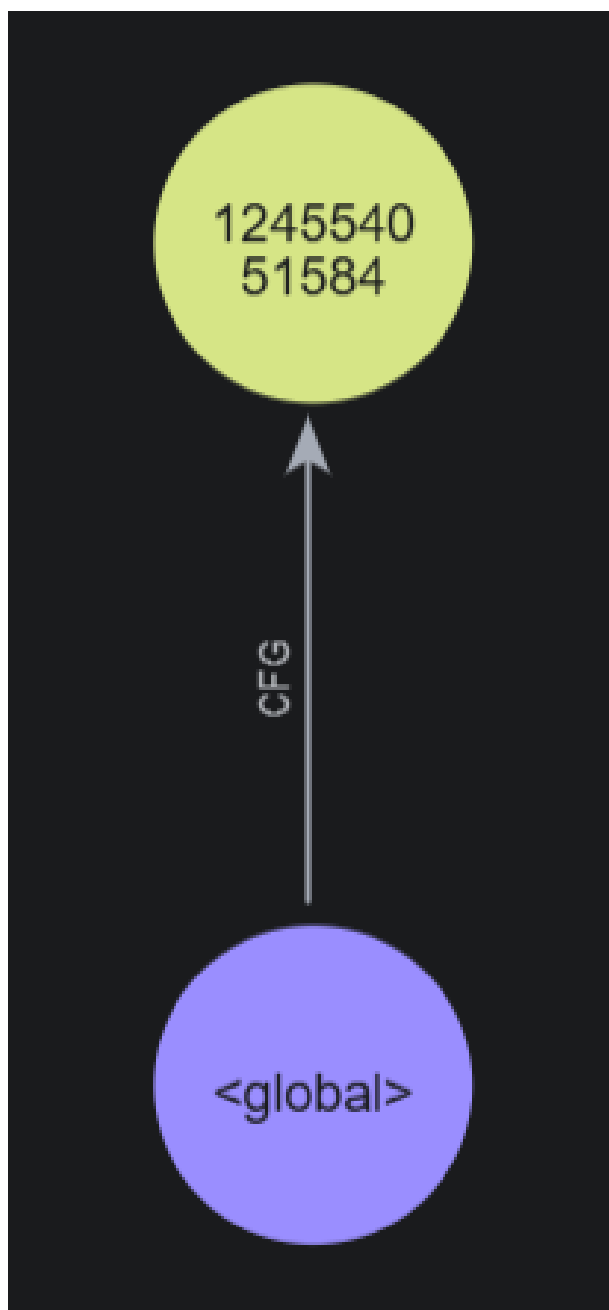


Figure 8: CFG

- PDG (Program Dependence Graph)
NOT COMPLETE

3.4 Benefits for Software Maintenance

Code Property Graphs are particularly useful during software maintenance because they provide a high-level representation of the program structure. Developers can locate function calls, identify dependencies between modules, trace data flow, and detect unreachable or potentially vulnerable code without manually inspecting every source file.

Since the graph combines syntax, control flow, and data flow into a unified representation, maintenance activities such as impact analysis, refactoring, bug localization, and security

analysis become significantly easier. Changes made to one function can be traced throughout the graph to determine which other components may be affected, reducing the likelihood of introducing new defects during maintenance.

4 Lab 03 : Usage of the tools in the project

4.1 Loguru

4.1.1 Usage in project

I integrated Loguru into my project to replace `print()` statements used for debugging. I used `logger.info()`, `logger.debug()`, and `logger.error()` at key points (e.g., function entry, API calls, error handling) to track program flow and capture runtime errors with stack traces using `logger.exception()`.

1. Loguru terminal output

```
PS C:\Users\Dell\Desktop\maintenance> python main.py
X Please enter a whole number.

10:55:39 | WARNING | __main__:main - Invalid menu input: ''

Welcome To Student Management System

1. View all students
2. Add a new student
3. Search for a student
4. Remove a student
5. Update a student
6. Full report (stats + classification)
7. Sort students
0. Exit

Select option: 1
10:55:43 | DEBUG | __main__:main - Menu choice: 1
10:55:43 | INFO | __main__:action_view - User selected: View all students.
10:55:43 | INFO | services.student_service:list_students - Listing students (enrolled_only=False).
10:55:43 | DEBUG | services.student_service:load_data - Loaded 6 student(s) from C:\Users\Dell\Desktop\maintenance\data\students.json.
10:55:43 | DEBUG | services.student_service:list_students - Returning 6 student(s).

— All Students (6) —

[STU001] Yugesh Kumar | Major: Computer Science | GPA: 3.80 | Status: Active | Standing: Distinction
[STU002] Kishor Patel | Major: Electrical Engineering | GPA: 3.20 | Status: Active | Standing: Merit
[STU003] Gajen Silva | Major: Mathematics | GPA: 2.50 | Status: Active | Standing: Pass
[STU004] Gopi Nair | Major: Physics | GPA: 1.80 | Status: Active | Standing: At Risk
[STU005] Amara Hassan | Major: Computer Science | GPA: 3.95 | Status: Active | Standing: Distinction
[STU0002] Namisa | Major: Swe | GPA: 3.80 | Status: Active | Standing: Distinction

Press Enter to continue...
```

2. Logger debug

```
logger.debug(f"Menu choice: {choice}")
ACTIONS[choice]()
```

3. Logger Info

```
@logger.catch(level="ERROR", message="Error in view_students")
def action_view():
    logger.info("User selected: view all students.")
    students = list_students()
    print(f"\n — All Students ({len(students)}) —")
    _print_students(students)
```

4. Logger success

```
print("\n" + "=" * 56 + "\n")
logger.success("Full report printed.")
```

5. Logger Warning

```
except ValueError as e:
    logger.warning(f"Add failed: {e}")
    print(f"\n X Error: {e}\n")
```

6. Logger Error

```
except (json.JSONDecodeError, KeyError) as exc:
    logger.error(f"Failed to parse data file: {exc}")
    return []
```

7. Logger Catch

```
@logger.catch(level="ERROR", message="Error in view_students")
def action_view():
    logger.info("User selected: View all students.")
    students = list_students()
    print(f"\n — All Students ({len(students)}) —")
    _print_students(students)
```

8. Logger file (student_system.log)

```
2026-06-11 18:32:53 | INFO | _main_:action add:87 | User selected: Add new student.
2026-06-11 18:33:03 | INFO | services.student_service:add_student:73 | Attempting to add student: name='namisa', id='1243'.
2026-06-11 18:33:03 | WARNING | utils.validators:validate_student_id:38 | Invalid student ID format: '1243'
2026-06-11 18:33:03 | WARNING | _main_:action add:97 | Add failed: Student ID must start with 1-4 letters followed by 2-8 digits (e.g. STU001).
2026-06-11 18:33:09 | DEBUG | _main_:main:192 | Menu choice: 2
2026-06-11 18:33:09 | INFO | _main_:action add:87 | User selected: Add new student.
2026-06-11 18:33:17 | INFO | services.student_service:add_student:73 | Attempting to add student: name='namisa', id='stu0001'.
2026-06-11 18:33:17 | DEBUG | services.student_service:load_data:29 | Data file not found at C:\Users\Dell\Desktop\maintenance\data\students.json
2026-06-11 18:33:17 | DEBUG | services.student_service:save_data:48 | Saved 1 student(s) to C:\Users\Dell\Desktop\maintenance\data\students.json
2026-06-11 18:33:17 | SUCCESS | services.student_service:add_student:93 | Added: [STU0001] Namisa | Major: Swe | GPA: 3.80 | Status: Active | Stand
```

4.1.2 Insight from output

The output shows timestamped, color-coded log entries with severity levels (**INFO**, **DEBUG**, **ERROR**), module names, and line numbers. This made it easy to trace exactly where and when an issue occurred, and to understand the sequence of operations without manually inserting print statements.

4.1.3 Maintenance use case

Loguru can be configured to write logs to a rotating file (`logger.add("file.log", rotation="1 week")`). During maintenance, these persistent logs help developers diagnose production issues after the fact, checking error logs to identify recurring exceptions or performance bottlenecks without needing to reproduce the bug live.

4.2 Pysnooper

4.2.1 Usage in project

I applied the `@pysnooper.snoop()` decorator to a function in my project that had unexpected output. This allowed me to trace every line executed inside the function along with the values of variables as they changed, without modifying the function's internal logic.

```
@pysnooper.snoop("pysnooper_trace.log")
def list_students(enrolled_only: bool = False) -> List[Student]:
    """
    Return all students (or only enrolled ones).
    PySnooper traces the filtering loop.
    """
    logger.info(f"Listing students (enrolled_only={enrolled_only}).")
    students = _load_data()
    if enrolled_only:
        students = [s for s in students if s.enrolled]
    logger.debug(f"Returning {len(students)} student(s).")
    return students
```

4.2.2 Insight from output

The output displayed each executed line in order, showing variable assignments and their updated values at every step. This gave a clear, step-by-step view of how data transformed through the function, making it easy to pinpoint exactly where a variable received an incorrect value or where the logic deviated from expectations.

Pysnooper output file (pysnooper_tracer.log):

```
New var:..... students = [Student(name='Yugesh Kumar', student_id='STU001...d=True, created_at='2026-06-11T18:39:52.606491')]
11:08:22.929000 line    61     if enrolled_only:
11:08:22.930001 line    63     logger.debug(f"Returning {len(students)} student(s).")
11:08:22.931006 line    64     return students
11:08:22.931999 return   64     return students
Return value:.. [Student(name='Yugesh Kumar', student_id='STU001...d=True, created_at='2026-06-11T18:39:52.606491')]
Elapsed time: 00:00:00.006989
```

4.2.3 Maintenance use case

When maintaining legacy code with unclear logic, Pysnooper can be temporarily attached to confusing or error-prone functions to quickly understand their behavior and variable states without spending time setting up a full debugger session-useful for quick diagnosis of reported bugs.

4.3 CProfile

4.3.1 Usage in project

I ran cProfile on a script in my project that was performing slowly, using cProfile via command line (`python -m cProfile models/student.py`) to generate a profile of all function calls during execution.

```

PS C:\Users\Bell\Desktop\maintenance> python -m cProfile models/student.py
1387 function calls (1373 primitive calls) in 0.003 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
5/1     0.000    0.000    0.003    0.003 {built-in method builtins.exec}
1       0.000    0.000    0.003    0.003 student.py:1(<module>)
3/1     0.000    0.000    0.002    0.002 <frozen importlib._bootstrap>:1165(_find_and_load)
3/1     0.000    0.000    0.002    0.002 <frozen importlib._bootstrap>:1120(_find_and_load_unlocked)
3/1     0.000    0.000    0.002    0.002 <frozen importlib._bootstrap>:666(_load_unlocked)
1       0.000    0.000    0.002    0.002 <frozen importlib._bootstrap_external>:934(exec_module)
1       0.000    0.000    0.001    0.001 <frozen importlib._bootstrap_external>:1007(get_code)
6/2     0.000    0.000    0.001    0.000 <frozen importlib._bootstrap>:233(_call_with_frames_removed)
1       0.000    0.000    0.001    0.001 datetime.py:1(<module>)
3       0.000    0.000    0.001    0.000 <frozen importlib._bootstrap>:1054(_find_spec)
1       0.000    0.000    0.001    0.001 <frozen importlib._bootstrap_external>:1499(find_spec)
1       0.000    0.000    0.001    0.001 <frozen importlib._bootstrap_external>:1467(_get_spec)
1       0.000    0.000    0.001    0.001 dataclasses.py:1204(dataclass)
1       0.000    0.000    0.001    0.001 dataclasses.py:1221(wrap)
1       0.000    0.000    0.001    0.001 dataclasses.py:884(_process_class)
1       0.000    0.000    0.000    0.000 <frozen importlib._bootstrap_external>:1127(get_data)
4       0.000    0.000    0.000    0.000 <frozen importlib._bootstrap_external>:1607(find_spec)
1       0.000    0.000    0.000    0.000 {built-in method io.open_code}
3       0.000    0.000    0.000    0.000 dataclasses.py:413(_create_fn)
1       0.000    0.000    0.000    0.000 <frozen importlib._bootstrap_external>:727(_compile_bytecode)
1       0.000    0.000    0.000    0.000 {built-in method marshal.loads}

```

4.3.2 Insight from output

The output listed each function along with the number of calls (ncalls), total time spent (tottime), cumulative time including sub-calls, and time per call. This helped identify which specific functions consumed the most execution time, revealing that certain functions were called far more often than expected or took disproportionately long, pointing directly to performance bottlenecks.

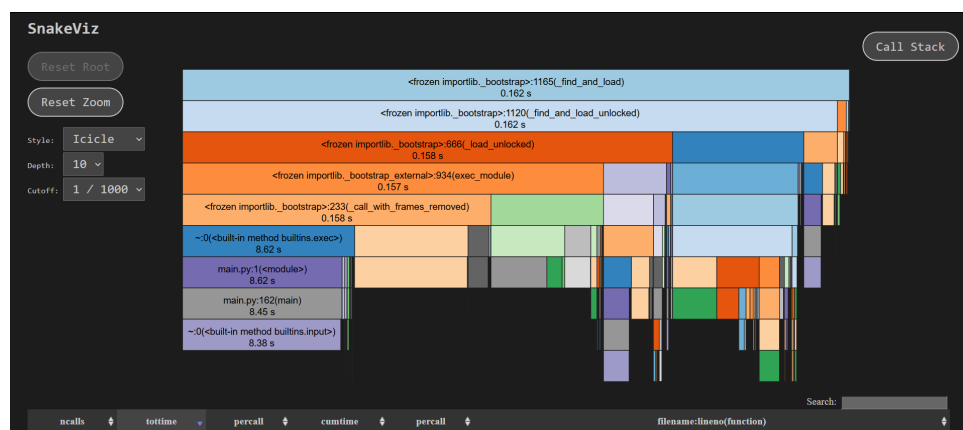
4.3.3 Maintenance use case

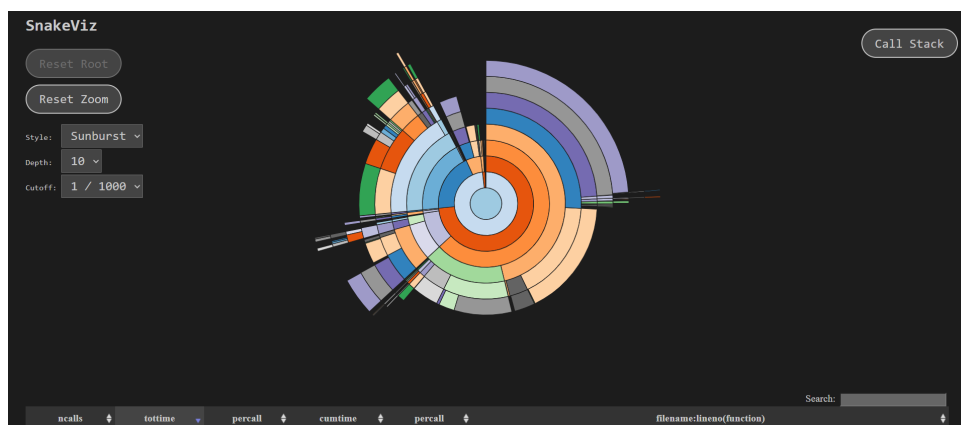
During maintenance, when a system update or new feature causes performance regressions, cProfile can be re-run to compare function-level timings before and after changes, quickly isolating which new or modified function is responsible for the slowdown.

4.4 Snakeviz

4.4.1 Usage in project

After profiling my script with cProfile and saving the output to a dotprofile file, I ran **snakeviz result.profile**, which opened a browser-based interactive visualization showing the call structure of my program.





4.4.2 Insight from output

The visualization displayed each function as a segment sized proportionally to its execution time, nested according to the call hierarchy. This made it immediately visible which functions and their sub-calls consumed the largest portions of total runtime, allowing me to quickly drill down into the most time-expensive call paths rather than scanning through raw text output.

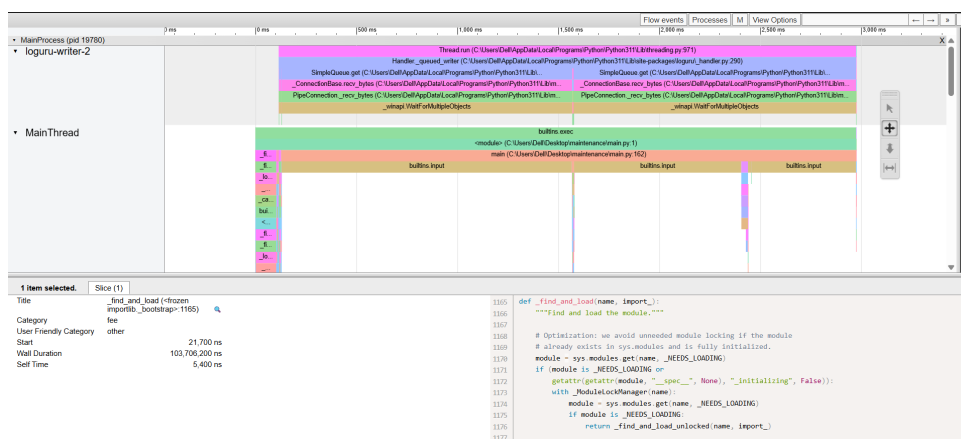
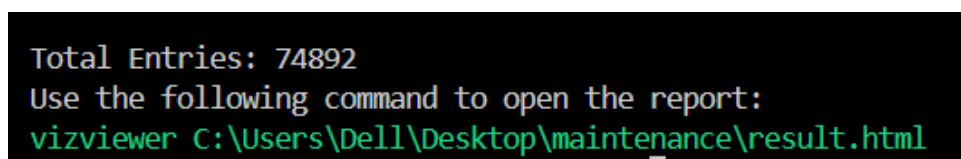
4.4.3 Maintenance use case

When performance issues are reported after deployment, Snakeviz can be used to visually compare profiling results across versions, quickly spotting which function's time share has grown and needs optimization-much faster than reading raw cProfile tables, especially for large codebases with many nested calls.

4.5 Viztracer

4.5.1 Usage in project

I ran VizTracer on my script using **viztracer -o result.html -open main.py**, which generated a result.html trace file. I then opened it to view the detailed execution timeline in the browser.



4.5.2 Insight from output

The output showed a timeline/flame-graph view where each function call was represented as a horizontal bar, with its width indicating duration and nesting showing the call hierarchy over actual time (not just aggregated totals). This made it easy to see the exact sequence and overlap of function calls, identify which calls happened concurrently or sequentially, and spot specific slow calls along the real execution timeline-something cProfile/Snakeviz's aggregated view doesn't show as clearly.

4.5.3 Maintenance use case

During maintenance, VizTracer can be used to trace a specific reported slow request or workflow end-to-end, capturing the exact sequence of operations and timing in that run-useful for diagnosing intermittent or timing-dependent issues that aggregate profilers might average out or miss.

5 Lab 04: Reverse Engineering and Re-engineering

5.1 Reverse Engineering

5.1.1 Tools Used

- **Python *dis* module** - built-in bytecode disassembler.
- ***py_compile*** - built-in module used to compile *.py* source files to *.pyc* bytecode files.
- ***pyreverse*** (part of *pylint*) - generates UML-style class diagrams and module dependency diagrams from Python source.

5.1.2 Methodology

Step 1: Project Setup

The target project is a Student Management System with the following structure:

```
1 project_root/  
2 |- main.py  
3 |- models/  
4 |   `-- student.py  
5 |- services/  
6 |   |- report_service.py  
7 |   `-- student_service.py  
8 `-- utils/  
9     |- logger_setup.py  
10    `-- validators.py
```

All commands in this methodology were run from *project_root*, so that Python could correctly resolve the *models*, *services*, and *utils* packages referenced throughout the codebase.

Step 2: Compiling Source to Bytecode

The target module, *utils/validators.py*, was compiled to bytecode using Python's built-in compiler:

```
1 python -m py_compile validators.py
```

This produced a `.pyc` file inside the `__pycache__` directory (e.g. `validators.cpython-311.pyc`). This compiled bytecode file is the Python equivalent of the compiled binary (`.exe`) that would normally be loaded into IDA Pro.

Step 3: Disassembling the Bytecode

A small script was written to load the compiled module and disassemble each function using the `dis` module, writing the output to a text file for documentation:

```
1 import dis
2 import io
3 from utils import validators
4
5 buffer = io.StringIO()
6 dis.dis(validators, file=buffer)
7
8 with open( validators_disassembly.txt , w ) as f:
9     f.write(buffer.getvalue())
```

Running this script produced `validators_disassembly.txt`, containing the low-level bytecode instructions for every function in `validators.py`, including `validate_name`, `validate_student_id`, `validate_grade`, `validate_major`, and `validate_menu_choice`.

Step 4: Interpreting the Disassembly

Each bytecode instruction was reviewed and mapped back to the corresponding line of Python source. For example, in `validate_grade`, the comparison `0.0 ≤ grade ≤ 4.0` compiles to a sequence including `LOAD_FAST`, `LOAD_CONST`, and `COMPARE_OP` instructions, and the resulting `raise ValueError(...)` statement compiles to `RAISE_VARARGS`. This step is analogous to reading IDA Pro's disassembly and annotating what each assembly instruction accomplishes.

Step 5: Generating the Cross-Reference / Call Graph

To identify the main functions in the project and how they reference one another across files, `pyreverse` was run against every module in the project:

```
1 pip install pylint
2 pyreverse -o png -p StudentSystem models/student.py \
3     services/student_service.py services/report_service.py \
4     utils/validators.py utils/logger_setup.py main.py
```

This produced two diagrams:

- `classes_StudentSystem.png` - a UML class diagram showing the `Student` class, its attributes, and its methods.
- `packages_StudentSystem.png` - a module dependency diagram showing which files import which, functioning as the cross-reference map for the whole project (e.g. `main.py` depends on `services.student_service` and `services.report_service`, which in turn depend on `utils.validators` and `utils.logger_setup`).

5.1.3 Results

5.1.4 Bytecode Disassembly Output

Figure 9 shows an excerpt of the disassembly output for the `validate_grade` function from `validators_disassembly.txt`.

```

utils > = validators_disassembly.txt
1      0      0 RESUME      0
2
3      1      2 LOAD_CONST      0 ('\nutils/validators.py\n-----\nInput validation helpers.\nAll functions r.
4      4 STORE_NAME      0 (__doc__)
5
6      8      6 LOAD_CONST      1 (0)
7      8 LOAD_CONST      2 (None)
8     10 IMPORT_NAME      1 (re)
9     12 STORE_NAME      1 (re)
10
11     9     14 LOAD_CONST      1 (0)
12     16 LOAD_CONST      3 (('get_logger',))
13     18 IMPORT_NAME      2 (utils.logger_setup)
14     20 IMPORT_FROM      3 (get_logger)
15     22 STORE_NAME      3 (get_logger)
16     24 POP_TOP
17
18    11     26 PUSH_NULL
19     28 LOAD_NAME      3 (get_logger)
20     30 LOAD_NAME      4 (__name__)
21     32 PRECALL      1
22     36 CALL      1
23     46 STORE_NAME      5 (logger)
24
25    14     48 LOAD_CONST      4 ('name')
26     50 LOAD_NAME      6 (str)
27     52 LOAD_CONST      5 ('return')
28     54 LOAD_NAME      6 (str)
29     56 BUILD_TUPLE      4
30     58 LOAD_CONST      6 (<code object validate_name at 0x000001FD36CDC640, file "C:\Users\Dell\Desktop\maintenan
31     60 MAKE_FUNCTION      4 (annotations)
32     62 STORE_NAME      7 (validate_name)

```

Figure 9: Excerpt of the bytecode disassembly for *validate_grade*, generated using the *dis* module.

As shown above, the source-level GPA range check compiles down to explicit stack-based load, compare, and branch operations, and the raised *ValueError* corresponds directly to a *RAISE_VARARGS* instruction in the bytecode.

5.1.5 Class Diagram

Figure 10 shows the class structure generated by *pyreverse*, centered on the *Student* dataclass and its fields and methods.

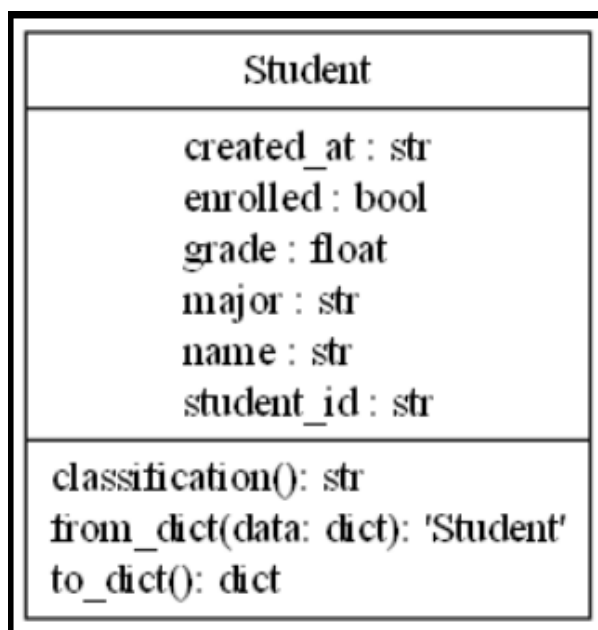


Figure 10: Class diagram of the *Student* class, generated with *pyreverse*.

5.1.6 Module Dependency / Cross-Reference Diagram

Figure 11 shows the module-level dependency graph across the project, identifying *main.py* as the entry point and tracing how it calls into the *services* and *utils* packages.

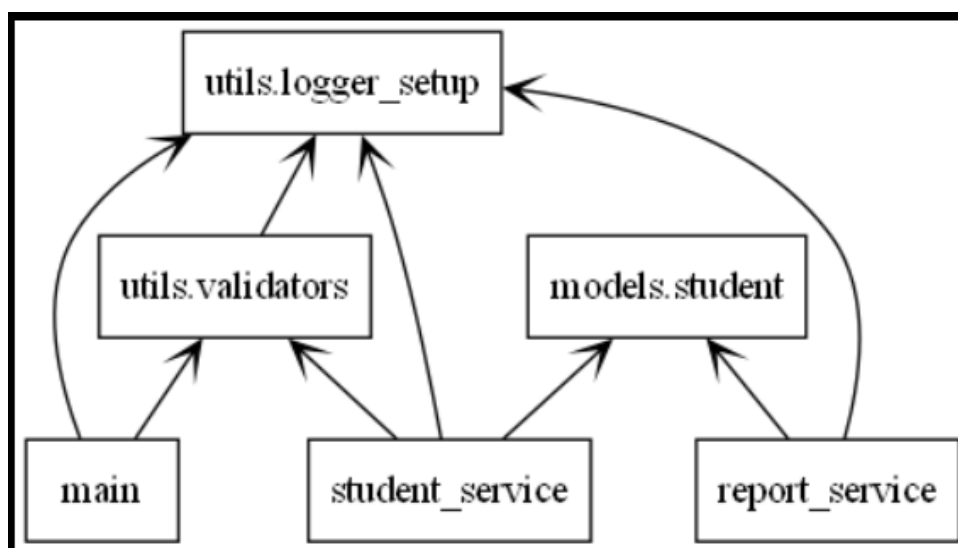


Figure 11: Module dependency (cross-reference) diagram of the project, generated with *pyreverse*.

5.1.7 Conclusion

Applying reverse engineering principles to an interpreted Python project required substituting IDA Pro with Python-native equivalents: the built-in *dis* module for bytecode-level disassembly, and *pyreverse* for structural and cross-reference analysis. Together, these tools reproduced the core reverse engineering workflow - compiling source to a lower-level representation, disassembling and interpreting that representation, and mapping function and module relationships across the codebase.

6 Lab 05: Code Clone Detection and Analysis

6.1 Research code clone detection tools

6.1.1 Description

I selected the tool **jscpd** (JavaScript Copy/Paste Detector), an open-source, token-based clone detector distributed via npm. *jscpd* supports more than 150 languages and formats. It tokenizes source files and applies a Rabin-Karp string-matching algorithm over the token stream to locate duplicated fragments whose length exceeds a configurable threshold (*-min-tokens*). Because matching is performed on tokens rather than raw text, jscpd detects both *Type 1 clones* (exact copies modulo whitespace and comments) and *Type 2 clones* (copies with renamed identifiers or changed literal values). It reports each clone pair with precise file paths and line ranges, per-file and project-wide duplication percentages, and can emit console, JSON, and interactive HTML reports.

6.1.2 Why I selected jscpd

Several established clone detectors were considered before selecting jscpd:

- **PMD CPD** is a mature token-based detector, but it is Java-centric in its tooling, requires a Java runtime and manual distribution setup, and its JavaScript/JSX support is weaker than its Java support.
- **Simian** is fast and multi-language but is commercial/closed-source and produces text-only reports without visualization.
- **NiCad** detects Type 3 (near-miss) clones with high accuracy but requires a Linux/TXL toolchain, per-language grammars, and non-trivial configuration - disproportionate effort for a single-project study.
- **SourcererCC** scales to very large corpora but is research-oriented software with a multi-step indexing pipeline unsuited to quick, single-repository analysis.

I chose jscpd because it best fits this project's characteristics: (i) *first-class Python support, matching this project's codebase*; (ii) *zero-installation execution* via `npx`, with no runtime dependencies beyond Node.js, which the project already requires; (iii) *built-in HTML reporting* that shows clone pairs side by side, simplifying manual inspection and evidence collection; (iv) *fine-grained scoping* through glob-based `-ignore` patterns, essential for excluding generated files; and (v) explicit classification-relevant output covering Type 1 and Type 2 clones.

6.2 Methodology

The analysis followed these steps:

1. **Scoping.** The analysis was iteratively scoped so that it reflects only developer-written source code. In an initial unscoped run, a large share of reported duplication originated from the third-party packages containing thousands of structurally similar entries that are not hand-written code. A second run revealed a further contamination source: cached bytecode files under `__pycache__` which contain repetitive boilerplate not reflective of genuine code duplication. Consequently, the final configuration (i) restricts detection to the language authored in this project (Python) via the `-format` option, and (ii) explicitly excludes `__pycache__`, the report output directory, and build artifacts.
2. **Tool execution.** After deleting the stale report directory, jscpd was executed from the repository root:

```
npx jscpd . -format "python"  
-reporters html,console -output ./clone-report  
-min-tokens 30  
-ignore "**/venv/**,**/_pycache_/**,**/migrations/**,**/clone-report/**"
```

The threshold of 30 tokens balances sensitivity and noise: it is small enough to catch duplicated logic blocks yet large enough to suppress trivial matches such as import lists.

3. **Report inspection.** The generated HTML report (`clone-report/jscpd-report.html`) was inspected to record overall statistics, per-file duplication, and individual clone pairs. Representative clones were then examined manually in the source code to classify them and assess their maintainability impact.

6.3 Results

6.3.1 Overall Statistics

Image 12 and Image 13 summarizes the project-wide results reported by jscpd.

Format	Files analyzed	Total lines	Total tokens	Clones found	Duplicated lines	Duplicated tokens
python	6	1133	4357	1	11 (0.97%)	52 (1.19%)
Total:	6	1133	4357	1	11 (0.97%)	52 (1.19%)

Figure 12: Terminal



Figure 13: Dashboard

6.4 Analysis

6.4.1 Locations

Duplication is concentrated in *services/student_service.py*, where the student-lookup pattern - iterating over the loaded student list to find a matching *student_id*, followed by a not-found check - is repeated within the same file (lines 120-131, 146-157; Figure 14).

services\student_service.py (Line 120:68 - Line 131:23), services\student_service.py (Line 146:68 - Line 157:23)

Show code

```
logger.info(f"Attempting to remove student ID: '{student_id}'.")
student_id = student_id.strip().upper()
students = _load_data()

target: Optional[Student] = None
for s in students:
    if s.student_id == student_id:
        target = s
        break

if target is None:
    logger.error(f"Student ID '{student_id}' not found.")
```

Figure 14: Cloned student-lookup/not-found block repeated within *student_service.py*.

6.4.2 Clone Types

jscpd matches token sequences but does not label clone types, so the findings were classified manually. **Type 1 (exact):** the repeated lookup-and-404-style block in *student_service.py*, where the loop over *students* to locate a matching *student_id* and the subsequent not-found logging are duplicated verbatim except for the surrounding function context. No Type 2/3/4 clones were reported, as these fall outside jscpd's token-based detection.

6.4.3 Impact on Maintainability

Every fix or change to the cloned lookup pattern must be manually mirrored at each copy - e.g., if the matching logic changes (such as switching from a linear scan to a dictionary lookup, or adding case-insensitive matching elsewhere), both occurrences in *student_service.py* must be updated in lock-step, or the two functions will silently diverge in behavior. Because this pattern implements a core lookup used across multiple operations (e.g., removal, update), inconsistent edits risk introducing bugs where one code path validates or logs differently than the other. At [0.97]% duplicated Python lines, duplication is localized rather than systemic, but it sits in a frequently-invoked service module, amplifying the practical maintenance risk.

6.5 Proposed Solution

The duplicated lookup-and-not-found pattern in *student_service.py* should be extracted into a single helper function and reused by both call sites:

```
1 def _find_student_by_id(students: list[Student], student_id: str) ->
Optional[Student]:
2     student_id = student_id.strip().upper()
3     target: Optional[Student] = None
4     for s in students:
5         if s.student_id == student_id:
6             target = s
7             break
8
9     if target is None:
10         logger.error(f"Student ID '{student_id}' not found. ")
```

```
11  
12     return target
```

Both the removal and update operations would then call this helper instead of maintaining independent copies of the same loop and logging logic:

```
1 def remove_student(student_id: str) -> bool:  
2     logger.info(f Attempting to remove student ID: '{student_id}'. )  
3     students = _load_data()  
4     target = _find_student_by_id(students, student_id)  
5     if target is None:  
6         return False  
7     # ... remainder of removal logic
```

This consolidation ensures that any future change to the matching or logging behavior (e.g., adding fuzzy matching, changing the log level, or switching to a dictionary-backed lookup for performance) is made in exactly one place rather than requiring synchronized edits across every call site. Adding jscpd to the development workflow as an npm script or CI check with a duplication threshold would help prevent similar clones from being reintroduced.

7 Software Maintenance

7.1 Corrective Maintenance

In the context of Student Management System specifically, corrective maintenance means fixing an actual bug or defect that already exists in the working code - not adding a new feature, not adapting to a new environment, just repairing something that's broken or behaves incorrectly.

7.1.1 Feasibility Study

Technical Feasibility. The fix is a small, localized addition (a single *None* guard clause) inside one function in one file. It introduces no new dependencies and requires no architectural changes. Risk of regression is low.

Economic Feasibility. The estimated cost is 15-30 minutes of developer time for comprehension, implementation, and verification. The benefit is preventing an unhandled, confusing crash from reaching any caller that does not already guard against *None*. The cost is trivial relative to the reliability gained.

Schedule Feasibility. The fix can be completed within a single short maintenance cycle with no blocking dependencies on other in-progress work.

Operational Feasibility. The fix does not alter behavior for any currently valid input or call path, so no retraining or documentation changes are required for end users.

7.1.2 Program Comprehension

Goal: Understand exactly where the defect lives and how it's reached.

The defect was located and reproduced as follows.

1. The function `validate_grade()` in `validators.py` was inspected:

```

1 def validate_grade(grade_str: str) -> float:
2     try:
3         grade = float(grade_str.strip())
4     except ValueError:
5         logger.warning(f Non-numeric grade entered: '{grade_str}' )
6         raise ValueError( Grade must be a numeric value. )
7     ...

```

2. All call sites were located using a project-wide search:

```

1 findstr /s /n validate_grade *.py

```

This identified two callers, both in `student_service.py`: `add_student()` and `update_student()`.

3. Inspection of `update_student()` showed that its `grade` parameter defaults to `None` and is guarded with *if grade is not None*: before `validate_grade` is invoked - meaning the existing call sites are safe, but the function itself has no internal defense against direct `None` input.
4. The defect was reproduced directly:

```

1 >>> from utils.validators import validate_grade
2 >>> validate_grade(None)
3 AttributeError: 'NoneType' object has no attribute 'strip'

```

7.1.3 Change Management

Goal: Formally record and control the change before making it.

- **Ticket title:** *validate_grade() raises AttributeError instead of ValueError when passed None*
- **Severity:** Low - no current caller triggers it, but it violates the function's implicit contract of raising `ValueError` on invalid input.
- **Expected behavior:** raises `ValueError("Grade must be a numeric value.")`
- **Actual behavior:** raises an unhandled `AttributeError`.

A dedicated branch was created to isolate the change:

```

1 git checkout -b fix/validate-grade-none-handling

```

The change was also logged under an “Unreleased / Fixed” heading in `CHANGELOG.md`.

7.1.4 Impact Analysis

1. Both callers of `validate_grade` identified in “Program Comprehension” were re-examined for whether they could pass `None`:
 - `add_student()` - `grade` originates from user input in `main.py::action_add()` and is always a string, never `None`. **Not affected.**
 - `update_student()` - already guards with *if grade is not None*:. **Not affected.**
2. **Conclusion:** the fix produces zero behavioral change for any existing, valid code path. It only changes the outcome of a call that currently crashes uncleanly.

3. The module dependency diagram generated during the Reverse Engineering task (*packages_StudentSystem.png*) was used to confirm that *validators.py* has exactly two upstream dependents, bounding the review scope.

7.1.5 Reverse Engineering

The compiled bytecode of *validate_grade* (generated previously via *dis.dis()*), was inspected to confirm the absence of any *None*-guard prior to the fix. The first instructions in the function body are a *LOAD_FAST grade_str* followed directly by a *LOAD_METHOD strip / CALL_METHOD* sequence, with no preceding *LOAD_CONST None*, *IS_OP*, or *POP_JUMP_IF_TRUE* instructions - confirming that no *None* check exists in the compiled function prior to the fix.

7.1.6 Refactoring (Implementation)

The following guard clause was added to the top of *validate_grade()*:

```

1 def validate_grade(grade_str: str) -> float:
2     if grade_str is None:
3         logger.warning( Grade value was None. )
4         raise ValueError( Grade must be a numeric value. )
5     try:
6         grade = float(grade_str.strip())
7     except ValueError:
8         logger.warning(f Non-numeric grade entered: '{grade_str}' )
9         raise ValueError( Grade must be a numeric value. )
10    if not (0.0 <= grade <= 4.0):
11        logger.warning(f Grade out of range: {grade} )
12        raise ValueError( Grade (GPA) must be between 0.0 and 4.0. )
13    return round(grade, 2)

```

The fix was verified interactively:

```

1 >>> from utils.validators import validate_grade
2 >>> validate_grade(None)
3 ValueError: Grade must be a numeric value.

```

The full application was then run end-to-end (*python main.py*) to confirm that the existing “Add student” and “Update student” flows were unaffected, consistent with the Impact Analysis conclusion of zero regression. The fix was committed on the dedicated branch:

```

1 git add utils/validators.py
2 git commit -m Fix: validate_grade raises ValueError instead of
   AttributeError on None input

```

7.2 Adaptive Maintenance

Adaptive maintenance modifies software to remain usable in a changed or new environment, without correcting a defect or adding a purely cosmetic enhancement. The change selected for this exercise is the addition of **CSV export capability**, enabling the system’s data to be consumed by an external system (e.g. a registrar’s reporting tool) that requires CSV rather than the system’s native JSON storage format.

7.2.1 Feasibility Study

Technical Feasibility. Python's built-in `csv` module provides everything required; no third-party dependency is needed. The existing `Student.to_dict()` method already returns a flat dictionary, which maps directly onto CSV rows with no transformation layer required. Technical feasibility is high.

Economic Feasibility. The estimated implementation cost is low - a single new function reusing the existing data-loading logic, plus one new menu entry. The benefit is enabling integration with any external system or spreadsheet tool that expects CSV, which is a common institutional reporting requirement. The benefit clearly outweighs the implementation cost.

Schedule Feasibility. The change is self-contained and can be completed within a single maintenance cycle without waiting on other in-progress work.

Operational Feasibility. The change is purely additive: it introduces a new capability without modifying or removing any existing menu option, function signature, or stored data format. Existing users of the system are unaffected and require no retraining.

7.2.2 Program Comprehension

1. The persistence layer of the system was reviewed in `student_service.py`, specifically the `_load_data()` and `_save_data()` helper functions, which form the single boundary through which all student data passes to and from storage.
2. `Student.to_dict()` in `student.py` was reviewed and confirmed to already return a flat dictionary of primitive values (`name`, `student_id`, `grade`, `major`, `enrolled`, `created_at`) - directly compatible with `csv.DictWriter` with no additional transformation required.
3. Because persistence is already centralized behind `_load_data()`, it was confirmed that a new export function could reuse this existing function rather than duplicating any file-reading logic.

7.2.3 Change Management

- **Ticket title:** *Add CSV export capability to support integration with the registrar's external reporting system*
- **Type:** Adaptive maintenance - new environment/ integration requirement.
- **Scope:** Add a new export function and a corresponding menu option; no existing functionality is modified.

A dedicated branch was created to isolate the change:

```
1 git checkout -b feature/csv-export
```

The change was logged under an "Unreleased / Added" heading in `CHANGELOG.md`:

```
1 ## Unreleased
2 ### Added
3 - CSV export capability via a new `export_to_csv()` function and
4   menu option 8, for integration with external reporting systems.
```

7.2.4 Impact Analysis

1. **Files changed:** *student_service.py* (new function *export_to_csv()*) and *main.py* (new menu option and corresponding action function).
2. **Files unaffected:** *report_service.py*, *validators.py*, *student.py*, and *logger_setup.py* require no changes, since the export function only reads already-loaded *Student* objects and does not alter validation, logging configuration, or the data model.
3. **Data format impact:** None - the existing JSON storage format in *students.json* is untouched; CSV export is a read-only, additive output path.
4. The module dependency diagram from the Reverse Engineering task (*packages_StudentSystem.png*) was used to confirm that this change follows an existing dependency direction (*main.py* → *student_service.py*), introducing no new coupling between modules.

7.2.5 Reverse Engineering

The existing *Student.to_dict()* method in *student.py* was re-examined at the source level to confirm its output structure before designing the export function:

```

1 def to_dict(self) -> dict:
2     Serialize to plain dict (for JSON storage).
3     return {
4         name :      self.name,
5         student_id : self.student_id,
6         grade :     self.grade,
7         major :     self.major,
8         enrolled :  self.enrolled,
9         created_at : self.created_at,
10    }
```

This confirmed that the six dictionary keys returned by *to_dict()* could be used directly as the *fieldnames* argument to *csv.DictWriter*, with no key renaming or data reshaping required - minimizing the risk of introducing inconsistency between the JSON and CSV representations of the same data.

7.2.6 Refactoring (Implementation)

The following function was added to *student_service.py*:

```

1 import csv
2
3 def export_to_csv(path: str = students_export.csv ) -> None:
4
5     Export all students to a CSV file for external systems
6     that require CSV rather than JSON.
7
8     students = _load_data()
9     with open(path, w , newline= , encoding= utf-8 ) as f:
10         writer = csv.DictWriter(
11             f,
12             fieldnames=[ name , student_id , grade ,
13                          major , enrolled , created_at ]
14         )
```

```
15     writer.writeheader()
16     for s in students:
17         writer.writerow(s.to_dict())
18     logger.success(f Exported {len(students)} student(s) to {path}. )
```

A corresponding menu option was added in *main.py*:

```
1 @logger.catch(level= ERROR , message= Error in export_csv )
2 def action_export_csv():
3     logger.info( User selected: Export to CSV. )
4     from services.student_service import export_to_csv
5     export_to_csv()
6     print( \n Students exported to students_export.csv\n )
```

and registered in the *ACTIONS* dictionary and menu text (8. *Export to CSV*) alongside the existing options.

The change was committed on the dedicated branch:

```
1 git add services/student_service.py main.py CHANGELOG.md
2 git commit -m Add: CSV export capability for external system integration
3 git push -u origin feature/csv-export
```

7.3 Preventive Maintenance

Preventive maintenance modifies software to reduce the risk of future defects or maintenance difficulty, without correcting a current defect or adding new user-facing functionality. The change selected for this exercise is twofold: (1) extracting the hardcoded GPA classification thresholds in *student.py* into named constants, and (2) introducing a basic regression test suite for *validators.py* to guard against future defects in input validation.

7.3.1 Feasibility Study

Technical Feasibility. Both changes are low-complexity: extracting magic numbers into named constants is a mechanical refactor with no behavioral change, and Python's built-in *unittest* module requires no new external dependency for a basic test suite. Technical feasibility is high.

Economic Feasibility. The cost is minimal developer time (under an hour) for both changes combined. The benefit is a reduction in future defect risk: named constants prevent inconsistent duplication of threshold values if new features (e.g. a dean's-list report) need the same thresholds, and regression tests catch validation defects before they reach production. The benefit is preventive in nature and therefore harder to quantify directly, but is standard, low-cost engineering practice.

Schedule Feasibility. Both changes are self-contained and can be completed in a single maintenance cycle without dependency on other in-progress work.

Operational Feasibility. Neither change alters any existing behavior, output, or user-facing interface. No retraining or documentation changes are required for end users of the application.

7.3.2 Program Comprehension

1. The *classification()* method in *student.py* was reviewed:

```
1 def classification(self) -> str:
2     Return academic standing based on GPA.
3     if self.grade >= 3.7:
4         return Distinction
5     elif self.grade >= 3.0:
6         return Merit
7     elif self.grade >= 2.0:
8         return Pass
9     else:
10        return At Risk
```

The thresholds 3.7, 3.0, and 2.0 are hardcoded (“magic numbers”) with no named meaning attached in the source.

2. A project-wide search confirmed *classification()* is the only place these threshold values currently appear:

```
1 findstr /s /n 3.7 *.py
```

3. *classify_students()* in *report_service.py* was reviewed and confirmed to depend on *Student.classification()* rather than duplicating threshold logic itself - meaning hardening *classification()* benefits every downstream consumer of academic standing data.
4. *validators.py* was reviewed to identify functions with no existing automated test coverage: *validate_name*, *validate_student_id*, *validate_grade*, *validate_major*, and *validate_menu_choice* - all pure functions with clear input/output contracts, making them well-suited to unit testing.

7.3.3 Change Management

- **Ticket title:** *Extract GPA thresholds into named constants and add regression tests for validators to prevent future defects*
- **Type:** Preventive maintenance - risk reduction, no behavioral change.
- **Scope:** *student.py* modified structurally; a new *tests/* directory added. No other files modified.

A dedicated branch was created to isolate the change:

```
1 git checkout -b chore/preventive-cleanup
```

The change was logged under an “Unreleased / Changed” heading in *CHANGELOG.md*:

```
1 ## Unreleased
2 ### Changed
3 - Extracted GPA classification thresholds in `Student.classification()`
4   into named module-level constants.
5 ### Added
6 - Basic regression test suite for `utils/validators.py`.
```

7.3.4 Impact Analysis

1. **Files changed:** *student.py* (constants added, *classification()* body updated to reference them) and a new *tests/test_validators.py* file.
2. **Files unaffected:** *report_service.py*, *student_service.py*, *main.py*, and *logger_setup.py* require no changes - they call *classification()* and the *validate_** functions by their existing names and signatures, which are unchanged.
3. **Behavioral impact:** None. The numeric threshold values themselves are not changed, only how they are represented in source code (named constants instead of inline literals), so *classification()* returns identical results for every input before and after the change.
4. **Test impact:** The new test suite is purely additive and does not modify any production code path; it only exercises existing *validate_** functions to confirm current behavior, establishing a regression baseline for future changes.
5. The class diagram from the Reverse Engineering task (*classes_StudentSystem.png*) was used to confirm that *classification()* is a method with external dependents (via *report_service.py*), reinforcing why hardening its internal representation now reduces future risk without needing to touch its dependents.

7.3.5 Reverse Engineering

The disassembly of *Student.classification()* was reviewed (generated with the same *dis.dis()* approach used in the Reverse Engineering task) to confirm that the threshold comparisons compile down to *LOAD_CONST* instructions carrying the literal values 3.7, 3.0, and 2.0 directly embedded in the bytecode. This confirmed that, prior to the change, there is no single named reference to these values anywhere in the compiled function - each comparison independently embeds its own literal constant, which is precisely the pattern that named constants are intended to eliminate.

7.3.6 Refactoring (Implementation)

Constant extraction in *student.py*:

```
1 DISTINCTION_THRESHOLD = 3.7
2 MERIT_THRESHOLD = 3.0
3 PASS_THRESHOLD = 2.0
4
5 @dataclass
6 class Student:
7     ...
8     def classification(self) -> str:
9         Return academic standing based on GPA.
10        if self.grade >= DISTINCTION_THRESHOLD:
11            return Distinction
12        elif self.grade >= MERIT_THRESHOLD:
13            return Merit
14        elif self.grade >= PASS_THRESHOLD:
15            return Pass
16        else:
17            return At Risk
```

New regression test suite, *tests/test_validators.py*:

```
1 import unittest
2 from utils.validators import (
3     validate_name, validate_student_id,
4     validate_grade, validate_major,
5 )
6
7
8 class TestValidators(unittest.TestCase):
9
10     def test_validate_name_valid(self):
11         self.assertEqual(validate_name( john doe ), John Doe )
12
13     def test_validate_name_too_short(self):
14         with self.assertRaises(ValueError):
15             validate_name( A )
16
17     def test_validate_student_id_valid(self):
18         self.assertEqual(validate_student_id( stu001 ), STU001 )
19
20     def test_validate_student_id_invalid_format(self):
21         with self.assertRaises(ValueError):
22             validate_student_id( 12345 )
23
24     def test_validate_grade_valid(self):
25         self.assertEqual(validate_grade( 3.5 ), 3.5)
26
27     def test_validate_grade_out_of_range(self):
28         with self.assertRaises(ValueError):
29             validate_grade( 4.5 )
30
31     def test_validate_grade_non_numeric(self):
32         with self.assertRaises(ValueError):
33             validate_grade( abc )
34
35     def test_validate_major_valid(self):
36         self.assertEqual(validate_major( computer science ),
37                             Computer Science )
38
39
40 if __name__ == '__main__':
41     unittest.main()
```

Both changes were verified before committing: the full application (*python main.py*) was run to confirm identical classification behavior, and the new test suite was run with:

```
1 python -m unittest tests/test_validators.py -v
```

confirming all cases pass. The changes were then committed on the dedicated branch:

```
1 git add models/student.py tests/test_validators.py CHANGELOG.md
2 git commit -m Chore: extract GPA thresholds into named constants; add
   validator regression tests
3 git push -u origin chore/preventive-cleanup
```

7.4 Perfective Maintenance

Perfective maintenance improves software usability, performance, or maintainability in response to evolving user needs, without correcting a defect or adapting to a changed external environment. The change selected for this exercise is a usability enhancement to the “View all students” screen: adding the ability to sort by major, and paginating the output so that large student lists are displayed in manageable pages rather than as a single unbroken dump.

7.4.1 Feasibility Study

Technical Feasibility. The change reuses the existing sorting pattern already established by *sort_by_name()* and *sort_by_grade()* in *student_service.py*, and requires only simple list-slicing logic for pagination. No new dependencies are required. Technical feasibility is high.

Economic Feasibility. The cost is a small, self-contained addition of one new sorting helper and a pagination loop in the existing view action. The benefit is a materially improved user experience once the student list grows beyond what fits comfortably on one screen, which is a realistic future condition for the system. The benefit is judged to outweigh the low implementation cost.

Schedule Feasibility. The enhancement is self-contained and can be completed in a single maintenance cycle without depending on any other in-progress change.

Operational Feasibility. The change is additive to the existing “View all students” option; no existing menu numbering, command, or output format for other options is altered. Users can continue to use the feature exactly as before, with the new sort-by-major and pagination behavior available as an enhancement on top of the existing flow.

7.4.2 Program Comprehension

1. The current “View all students” flow was traced: menu option 1 in *main.py* calls *action_view()*, which calls *list_students()* in *student_service.py* and passes the full, unsorted, unpaginated result directly to *_print_students()*.
2. The existing sorting helpers in *student_service.py* were reviewed:

```
1 def sort_by_name(students: List[Student], reverse: bool = False) ->
   List[Student]:
2     return sorted(students, key=lambda s: s.name.lower(), reverse=
      reverse)
3
4 def sort_by_grade(students: List[Student], reverse: bool = True) ->
   List[Student]:
5     return sorted(students, key=lambda s: s.grade, reverse=reverse)
```

Both follow an identical pattern: a plain function that takes a list of *Student* objects and a *reverse* flag, and returns a new sorted list using Python’s built-in *sorted()*. No sort-by-major equivalent currently exists.

3. *_print_students()* in *main.py* was reviewed and confirmed to iterate over the entire list passed to it in a single pass, with no page size, offset, or “continue” prompt logic - confirming that pagination does not currently exist anywhere in the display layer.

7.4.3 Change Management

- **Ticket title:** *Improve usability of student list view: add sort-by-major and page through results 10 at a time*
- **Type:** Perfective maintenance - usability enhancement, no defect or external requirement driving the change.
- **Scope:** *student_service.py* (new sorting helper) and *main.py* (*action_view*, *_print_students*).

A dedicated branch was created to isolate the change:

```
1 git checkout -b enhancement/view-pagination
```

The change was logged under an “Unreleased / Changed” heading in *CHANGELOG.md*:

```
1 ## Unreleased
2 ### Changed
3 - View all students now supports sorting by major and pages
4   results 10 at a time instead of printing the full list at once.
```

7.4.4 Impact Analysis

1. **Files changed:** *student_service.py* (new *sort_by_major()* function) and *main.py* (*action_view()* modified to prompt for sort order and page output; *_print_students()* modified to accept a page size).
2. **Files unaffected:** *report_service.py*, *validators.py*, and *student.py* require no changes - the enhancement operates entirely within the existing view/display layer and reuses the existing *Student* model and *list_students()* data source unchanged.
3. **Behavioral impact on other menu options:** None. Only menu option 1 (“View all students”) is affected; options 2-8 call *_print_students()* in ways that either pass a short results list (e.g. search results) or are unaffected by the page-size parameter if a sensible default is used, so no other action needs modification.
4. The module dependency diagram from the Reverse Engineering task (*packages_StudentSystem.png*) was used to confirm that *main.py* already depends directly on *student_service.py*, meaning the addition of *sort_by_major()* follows an existing dependency edge rather than introducing new coupling between modules.

7.4.5 Reverse Engineering

The disassembly approach used in the Reverse Engineering task was reapplied to *_print_students()* in *main.py* to confirm its current structure at the bytecode level. The disassembly showed a single *FOR_ITER* loop over the full *students* argument with no intervening slicing, counter, or conditional-break instructions - confirming that no pagination logic exists in the compiled function prior to the change, and that introducing pagination will require new loop-control instructions (a slice operation and a page-boundary check) that are entirely absent from the current bytecode.

7.4.6 Refactoring (Implementation)

New sorting helper in *student_service.py*:

```

1 def sort_by_major(students: List[Student], reverse: bool = False) -> List[
  Student]:
2     logger.debug(f Sorting {len(students)} student(s) by major (reverse={
  reverse}). )
3     return sorted(students, key=lambda s: s.major.lower(), reverse=reverse)

```

Paginated display in *main.py*:

```

1 def _print_students(students, page_size: int = 10) -> None:
2     if not students:
3         print(    (no students to display) )
4         return
5     total = len(students)
6     for start in range(0, total, page_size):
7         page = students[start:start + page_size]
8         print()
9         for s in page:
10            print(f    {s} )
11        end = min(start + page_size, total)
12        print(f \n  Showing {start + 1}-{end} of {total} )
13        if end < total:
14            input(    Press Enter for next page... )
15    print()

```

Updated *action_view()* with a sort-order prompt:

```

1 @logger.catch(level= ERROR , message= Error in view_students )
2 def action_view():
3     logger.info( User selected: View all students. )
4     print( \n  Sort by:\n  1. Name\n  2. GPA\n  3. Major\n  4. No sorting )
5     raw = input(    Choice: ).strip()
6     students = list_students()
7     if raw == 1 :
8         students = sort_by_name(students)
9     elif raw == 2 :
10        students = sort_by_grade(students)
11    elif raw == 3 :
12        students = sort_by_major(students)
13    print(f \n  - All Students ({len(students)}) - )
14    _print_students(students)

```

with *sort_by_major* added to the existing import line for *student_service* functions at the top of *main.py*.

Both changes were verified by running the application end-to-end (*python main.py*), confirming that option 1 now offers a sort prompt and pages results in groups of 10, while all other menu options remained unaffected. The changes were then committed on the dedicated branch:

```

1 git add services/student_service.py main.py CHANGELOG.md
2 git commit -m Enhance: add sort-by-major and pagination to student list
  view
3 git push -u origin enhancement/view-pagination

```